

Big Data Lab

Lab 3: Apache Iceberg, Table Formats & Data Governance

Dr. Innocent Nyalala

IIT Madras Zanzibar

Department of Data Science and AI

Monday, 23 March 2026

DO.jpg

📖 Theory (2:00–3:00)

- **Core:** Why table formats exist
- **Architecture:** Manifests & Snapshots
- **Evolution:** Hidden partitions & schema
- **Features:** Time Travel & Rollback
- **Governance:** Fundamentals

💻 Hands-On (3:15 –4:30)

- **Setup:** SparkSession & MinIO
- **Action:** Create Iceberg tables
- **Practice:** Partition & Schema updates
- **Time Travel:** History & Live rollbacks
- **Comparison:** Iceberg vs. Delta Lake



15-Minute Break at 3:00 PM



Deadline Today: Submit team sheet on Classroom by **5:30 PM**.

DO.jpg

Quick Recap: Delta Lake from Lab 2

✓ Mastered in Lab 2

- **Logs:** `_delta_log/` JSON commits
- **ACID:** Reliable atomic writes
- **Time Travel:** `VERSION / TIMESTAMP`
- **MERGE/UPSERT:** Atomic modifications
- **Schema:** Enforcement & `mergeSchema`
- **Performance:** `OPTIMIZE` & `ZORDER`

❗ Common Mistakes

- **Docker:** Use `minio:9000` (not `localhost`)
- **Config:** Missing `spark.sql.extensions`
- **Format:** `MERGE` on non-Delta paths
- **History:** Querying lost snapshots post-`RESTORE`
- **Safety:** `VACUUM` without a `DRY RUN`

DO.jpg

The Problem: Plain Files Break at Petabyte Scale

The 2015 "Data Lake" Setup

- **Storage:** Parquet files in S3/MinIO buckets
- **Tracking:** Hive Metastore maps tables to directories
- **Scale:** Works fine for thousands of files

Netflix's Breaking Point (2017)

- **Slow Listings:** 1M+ files per table → Hive took 10 mins just to **find** files
- **No Atomic Commits:** Readers saw partial data during writes
- **Fragile Schema:** Column renames required rewriting the whole table
- **Leaky Abstractions:** Users had to query physical layouts e.g., `WHERE year=2024`

*Hive tracked **folders**. Netflix needed to track **individual files**.*

DO.jpg

The Solution: What is a Table Format?

A specification that sits **between** your physical storage (MinIO/S3) and your query engine (Spark/Trino).

Core Capabilities

- **True ACID:** Transactions without a DB server
- **O(1) RPC Calls:** Efficient metadata scanning instead of slow S3 file listings
- **Engine Agnostic:** Works with Spark, Flink, Trino simultaneously

Evolution & Safety

- **Schema Evolution:** Add, drop, rename safely
- **Partition Evolution:** Change partitioning rules without rewriting data
- **Time Travel:** Query historical states instantly

DO.jpg

Visualizing the Shift: Plain Files vs. Table Formats

architecture_comparison.png

DO.jpg

The Open Table Format Wars: Delta, Iceberg, Hudi

Feature	 Delta Lake	 Iceberg	 Hudi
Creator	Databricks	Netflix/Apache	Uber/Apache
Metadata	JSON commit log	Avro manifests	Avro timeline
Schema Evol.	Good	Excellent	Good
Partition Evol.	No	Yes (no rewrite)	No
Ecosystem	Spark primary	Native all engines	Spark primary
Updates	Excellent	Good	Excellent
Streaming	Excellent	Good	Excellent
Best For	Spark/Databricks	Multi-engine cloud	Low-latency

⚡ **Industry Direction (Apache XTable):** Universal interoperability is coming.

Why Iceberg today? We use Delta as our primary, but Iceberg shows the full open ecosystem.

DO.jpg

Comparing the 'Big 3' Open Table Formats

sessions/lab03/slides/ecosystems_map.png

DO.jpg

Apache Iceberg: The Paradigm Shift

💡 Core Design Decisions

- **File-Level Tracking:** $O(1)$ metadata lookups; eliminates slow S3 LIST calls.
- **Immutable Column IDs:** Renames update the metadata, not the Parquet files.
- **Hidden Partitioning:** Users query logically; the engine handles physical mapping.
- **Snapshot Isolation:** Consistent reads; readers never see partial data writes.
- **Open Spec:** A specification, not just a library (avoids vendor lock-in).

🌐 Global Adoption

- **Netflix:** The creator (100s of PBs)
- **Apple:** Exabyte-scale analytics
- **Amazon:** Native in Athena/Glue
- **LinkedIn:** 1000s of prod tables
- **Others** Uber, Expedia, Dremio, Adobe

Status: Apache Top-Level Project (Since 2020) | **Current:** Version 1.5

Compatibility: Native support for Spark 3.x, Flink, Trino, Hive, Dremio, Impala

DO.jpg

Apache Iceberg: Logical Architecture

logical_architecture.png

DO.jpg

Iceberg Architecture: Four Metadata Layers

four_layers.png

DO.jpg

How Iceberg Avoids Expensive S3 File Listings

Traditional Hive Read Path:

1. Call S3 LIST on the table directory.
2. At 1M+ files, this takes **minutes**.
3. Read files matching the partition filter.
4. Apply WHERE clause *after* reading data.

Iceberg Read Path (No S3 LIST):

1. Read `metadata.json` (instant).
2. Read manifest list (small Avro file).
3. Use **min/max stats** to skip irrelevant files.
4. Read *only* the necessary Parquet files.

Data Skipping via Statistics

Each manifest file records stats for the data files it references:

- File path on object storage
- **Min & Max value** per column
- Null count per column
- Total row count

The Magic (Data Skipping):

A query `WHERE ts > '2026-03-01'` automatically skips any file where `max(ts) < '2026-03-01'`.

Result: Zero Parquet files opened unnecessarily!

00.jpg

How Iceberg Avoids Expensive S3 File Listings

sessions/lab03/slides/hive_vs_iceberg.png

DO.jpg

Hidden Partitioning: Users Never Write Partition Columns

The Problem: Traditional Hive

- Requires **physical partition columns** to be created (e.g., `year INT, month INT`).
- Queries **must match** the physical layout:
`WHERE year=2026 AND month=3`
- If you change the scheme (e.g., `year/month` → `year/week`), you must **rewrite all historical data**.
- Analysts must memorize how the table is physically organized on disk.

The Solution: Iceberg Hidden Partitioning

- Partition specs use **transform functions** on actual data columns e.g.,
`months(event_time)`.
- Users query the **actual columns**:
`WHERE event_time >= '2026-03-01'`
- Iceberg's metadata maps this filter to the correct physical files **automatically**.
- Analysts don't need to know the physical layout!

Partition Transform Functions & Evolution

Available Transform Functions:

Transform	Description	Physical Folder Example
<code>year(ts)</code>	Extract year	<code>ts_year=2026</code>
<code>month(ts)</code>	Extract month	<code>ts_month=2026-03</code>
<code>day(ts)</code>	Extract day	<code>ts_day=2026-03-23</code>
<code>hour(ts)</code>	Extract hour	<code>ts_hour=2026-03-23-14</code>
<code>bucket(N, col)</code>	Hash into N buckets	<code>id_bucket=3</code>
<code>truncate(L, str)</code>	Prefix truncation (L chars)	<code>name_trunc=smi</code>

The Ultimate Benefit: Partition Evolution

Because user queries only reference the actual data column (`event_time`) and not the physical partition, you can seamlessly change your partition scheme in the future e.g., moving from daily to hourly partitioning as data grows. **Old queries will still work perfectly** across both the old and new data layouts!

DO.jpg

Partition Evolution: From Weeks to Milliseconds

Legacy (Hive/Parquet)

- **Hard-coded Paths:** Partitioning is tied to the folder structure.
- **The Cost:** Changing from Month → Day requires a **Full Table Rewrite**.
- **Downtime:** Table is locked during migration.

```
# THE OLD COSTLY WAY:  
INSERT INTO events_v2  
SELECT * FROM events_v1; -- PETABYTES  
DROP TABLE events_v1;
```

The Iceberg Revolution

- **Logical Mapping:** Partitioning is a metadata instruction, not a folder.
- **Metadata-Only:** One SQL command updates the "Spec Map."
- **Zero Rewrite:** Old files stay; new files adapt.

```
-- THE ICEBERG WAY:  
ALTER TABLE prod.events  
REPLACE PARTITION FIELD months(ts)  
WITH days(ts); -- MICROSECONDS
```

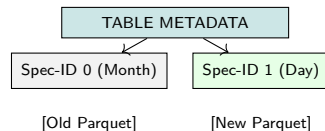
DO.jpg

Under the Hood: The Partition Spec ID

Each Iceberg data file records its **partition spec ID** in the manifest.

How it avoids the rewrite:

- **Spec-ID 0:** Old files marked as "Month-based."
- **Spec-ID 1:** New files marked as "Day-based."
- **Hybrid Planning:** The Query Engine reads *both* simultaneously.



"At Apple, this saved weeks of compute on exabyte-scale tables where rewrites were physically impossible."

Result: Evolution is a *pointer change*, not a *data move*.

DO.jpg

Schema Evolution: The Column ID Innovation

The Hive Problem:

- Parquet identifies columns by **string name**.
- Rename amount → txn_amt: Reader finds *nothing* in old files.
- **Result:** Full table rewrite required.

The Iceberg Innovation:

- Every column gets a **Permanent ID** (e.g., ID: 5).
- Parquet files store **ID 5**, not the name.
- **Rename:** Update metadata.json only.

METADATA: ID:5 → "pm2_5"



STORAGE: Parquet stores [ID:5]

```
# RENAME (ZERO data I/O)
spark.sql("""
    ALTER TABLE local.db.sensors
    RENAME COLUMN pm25 TO pm2_5
""")

# REORDER
spark.sql("""
    ALTER TABLE local.db.sensors
    ALTER COLUMN battery_level FIRST
""")

# DROP
spark.sql("""
    ALTER TABLE local.db.sensors
    DROP COLUMN old_field
""")

# Note: Data remains physically;
# metadata simply stops tracking ID.
```

Schema Evolution: Iceberg vs Delta Lake

Operation	Delta Lake 3.x	Iceberg 1.5
Add column	Yes (mergeSchema)	Yes
Rename column	Limited (Spark 3.4+)	Yes, zero I/O
Reorder columns	No	Yes
Drop column	No (column mapping)	Yes
Widen type (int→bigint)	Limited	Yes
Column comments	Yes	Yes
How columns tracked	By name (string)	By integer ID
Rename cost	Rewrite data files	Metadata only

Key Insight: Iceberg's **permanent field IDs** decouple a column's identity from its name. While Delta is catching up, Iceberg's architecture was built for metadata-only renames from Day 1.

Snapshot Management and Time Travel

Snapshots Lineage

- **Write Ops:** Append, Delete, Overwrite create a new Snapshot.
- **Immutability:** Old data stays until *Expired*.

Inspect History

```
-- Query the snapshot table
SELECT snapshot_id, operation,
       summary['total-records'] AS count
FROM local.db.sensors.snapshots;
```

Read Time Travel

```
# Read by Snapshot ID
df = spark.read.option("snapshot-id", 123) \
    .format("iceberg").load("db.sensors")

# Read by Timestamp
df = spark.read.option("as-of-timestamp", ts) \
    .format("iceberg").load("db.sensors")
```

Rollback Expiry

```
-- Atomic Rollback to ID
CALL system.rollback_to_snapshot('t', ID);

-- Expire (Vacuum)
CALL system.expire_snapshots('t', 3);
```

Delta vs. Iceberg Syntax:

VERSION AS OF (Delta) vs snapshot-id (Iceberg)
VACUUM (Delta) vs expire_snapshots (Iceberg)

Data Compaction: Solving the "Small File" Problem

The Streaming Penalty:

- Frequent commits create thousands of tiny Parquet files.
- **Result:** Slow metadata scans and high S3/HDFS overhead.

The Iceberg Solution:

- **Rewrite Data Files:** Merges small files into optimal 128MB/512MB chunks.
- **Zero Downtime:** Performed as an atomic transaction while readers continue.

Spark Maintenance

```
# MERGE small files into large ones
spark.actions(table) \
  .rewrite_data_files() \
  .target_file_size_bytes(128 * 1024 * 1024) \
  .execute()

# REWRITE Manifests
# (Optimizes the metadata itself)
spark.actions(table) \
  .rewrite_manifests() \
  .execute()
```

Optimization Tip: Run compaction during low-traffic hours to keep your "Time Travel" history clean and your queries

Iceberg Catalogs: Discovery & Atomic Commits

The Catalog Problem:

- **Discovery:** Iceberg metadata is just files on S3. How does Spark know which `metadata.json` is the "current" one?
- **Atomicity:** Two writers can't both update the table at once. The Catalog acts as the **Lock/Single Source of Truth**.
- **Mapping:** It maps `db.table` → `s3://path/to/metadata.json`.

Catalog	How it handles Commits
Hadoop	Uses Atomic Renames in the File System.
Hive	Uses Hive Metastore (HMS) DB locks.
REST	Uses an API (Nessie/Polaris) for commits.
Glue	Uses AWS DynamoDB/Optimistic locking.

Lab: Hadoop Catalog

`s3a://warehouse/iceberg/`

```
sensors/  
  metadata/  
    v1.metadata.json  
    v2.metadata.json <-- New Pointer  
  snap-101.avro  
data/  
  month=2026-03/  
    001.parquet
```

The "Magic" Link:

The Catalog tells Spark: "The current state is `v2.metadata.json`".

DO.jpg

Project Nessie: Git-like Version Control for Data

Transactional Catalog Operations:

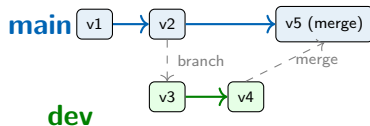
- **Branching:** Create `etl-test` from `main`. Test new logic without affecting production users.
- **Atomic Multi-Table Commit:** Merge changes across 10+ tables simultaneously.
- **Tagging:** Create a "Golden Snapshot" for reproducible ML model training (tag: `v1.0-model`).
- **Revert:** Instantly undo a "bad" batch job by resetting the branch pointer.

Nessie CLI / SQL

```
-- Create a branch for experimentation
CREATE BRANCH dev_experiment IN nessie;

-- Merge validated data back to main
MERGE BRANCH dev_experiment INTO main IN nessie;
```

Visualizing the Commit Graph



```
# Spark Configuration
spark.sql.catalog.nessie.uri=http://nessie:19120
spark.sql.catalog.nessie.ref=main
```

Nessie enables WAP (Write-Audit-Publish) workflows: write to branch, audit data quality, then publish to main.

DO.jpg

Data Governance: The Five Pillars

Why Governance Matters:

- **Discoverability:** Can I find the "official" table?
- **Security:** Can I block unauthorized PII access?
- **Quality:** Is the data 3 days old or fresh?

Our Implementation Stack:

- **Catalog:** Iceberg + Nessie (Lineage/History)
- **Quality:** Great Expectations (Lab 13)
- **Access:** MinIO Policy + Unity Catalog

The Governance Pillars:

1. **Discovery:** Table metadata & owners.
2. **Lineage:** Where did this cell come from?
3. **Quality:** Validation gates alerts.
4. **Access:** Role-based control (RBAC).
5. **Retention:** GDPR "Right to forget."

Governance is not a tool; it is the process of ensuring data is trustworthy and compliant.

Hands-On: SparkSession Config for Iceberg

```
from pyspark.sql import SparkSession

# Config: Downloads runtime JAR (~25MB) and enables Iceberg Extensions
spark = SparkSession.builder.appName("Lab03-Iceberg") \
    .config("spark.jars.packages", "org.apache.iceberg:iceberg-spark-runtime-3.5_2.12:1.5.2") \
    .config("spark.sql.extensions", "org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions") \
    .config("spark.sql.catalog.local", "org.apache.iceberg.spark.SparkCatalog") \
    .config("spark.sql.catalog.local.type", "hadoop") \
    .config("spark.sql.catalog.local.warehouse", "s3a://warehouse/iceberg") \
    .config("spark.hadoop.fs.s3a.endpoint", "http://minio:9000") \
    .config("spark.hadoop.fs.s3a.access.key", "minio") \
    .config("spark.hadoop.fs.s3a.secret.key", "minio123") \
    .config("spark.hadoop.fs.s3a.path.style.access", "true") \
    .config("spark.hadoop.fs.s3a.impl", "org.apache.hadoop.fs.s3a.S3AFileSystem") \
    .getOrCreate()

# Initialize Workspace
spark.sql("CREATE NAMESPACE IF NOT EXISTS local.lab03")

# Verify Connection
print(f"Spark: {spark.version} | Catalog: {spark.conf.get('spark.sql.catalog.local.warehouse')}")
```

Note: We use the Hadoop catalog type here to store metadata directly alongside data in MinIO without needing an external database server.

DO.jpg

Hands-On Part 2: Writing Your First Iceberg Table

```
import pandas as pd, numpy as np
from datetime import datetime, timedelta

# 1. Generate Synthetic Environmental Data (10k rows)
stations = ["Zanzibar-Airport", "Dar-Port", "Nairobi-CBD", "Kampala-Kololo", "Arusha-Summit"]
n = 10000
data = {
    "sensor_id": [f"SEN{r:04d}" for r in np.random.randint(1, 201, n)],
    "station": np.random.choice(stations, n),
    "temp": np.round(np.random.normal(28, 5, n), 2),
    "pm25": np.round(np.random.exponential(18, n), 1),
    "recorded_at": [datetime(2026, 1, 1) + timedelta(hours=int(h)) for h in np.random.randint(0, 2160, n)],
    "quality": np.random.choice(["GOOD", "WARN", "BAD"], n, p=[0.88, 0.09, 0.03])
}
df = spark.createDataFrame(pd.DataFrame(data))

# 2. Write as Iceberg with Hidden Partitioning
# Partitioned by MONTH transform -- no extra column needed in schema!
df.writeTo("local.lab03.sensors") \
    .partitionedBy(months("recorded_at")) \
    .createOrReplace()

# 3. Verify Table
print(f"Rows: {spark.table('local.lab03.sensors').count()}")
spark.table("local.lab03.sensors").printSchema()
```

DO.jpg

Hands-On Part 3: Partition + Schema Evolution

```
# ---- PARTITION EVOLUTION ----
# Change from months() to days() -- METADATA ONLY, no data rewrite
spark.sql("""
    ALTER TABLE local.lab03.sensors
    REPLACE PARTITION FIELD months(recorded_at) WITH days(recorded_at)
""")
print("Partition evolved to daily.")

# Append new data -- will use the NEW (daily) partition spec
df.sample(fraction=0.05, seed=99).writeTo("local.lab03.sensors").append()

# ---- SCHEMA EVOLUTION ----
# Add a new column
spark.sql("""
    ALTER TABLE local.lab03.sensors
    ADD COLUMN battery_level DOUBLE COMMENT 'Sensor battery percent 0-100'
""")

# Rename a column (ZERO data I/O -- only metadata.json changes!)
spark.sql("ALTER TABLE local.lab03.sensors RENAME COLUMN pm25 TO pm2_5")
print("Schema evolved: added battery_level, renamed pm25->pm2_5")

# Verify: old rows have NULL in battery_level (expected)
spark.sql("""
    SELECT sensor_id, station, pm2_5, battery_level
    FROM local.lab03.sensors
    WHERE battery_level IS NULL LIMIT 3
""").show()
```

DO.jpg

Hands-On Part 4: Snapshots, Time Travel and Rollback

```
# 1. View Snapshot Lineage
spark.sql("""
    SELECT snapshot_id, committed_at, operation,
           summary['added-records'] AS added, summary['total-records'] AS total
    FROM local.lab03.sensors.snapshots ORDER BY committed_at
""").show(truncate=False)

# 2. Extract First Snapshot ID for Time Travel
first_snap = spark.sql(
    "SELECT snapshot_id FROM local.lab03.sensors.snapshots ORDER BY committed_at LIMIT 1"
).first()["snapshot_id"]

# 3. Time Travel: Read state before Schema Evolution
df_v0 = spark.read.option("snapshot-id", first_snap) \
    .format("iceberg").load("local.lab03.sensors")
print(f"Snapshot 0: {df_v0.count()} rows, columns: {df_v0.columns}")

# 4. Disaster Recovery: Rollback the entire table to Snapshot 0
spark.sql(f"CALL local.system.rollback_to_snapshot('lab03.sensors', {first_snap})")
print("After rollback:", spark.table("local.lab03.sensors").count(), "rows")

# 5. Maintenance: Expire old snapshots to reclaim storage
spark.sql("CALL local.system.expire_snapshots(table=>'lab03.sensors', retain_last=>3)")
```

- **Snapshot Isolation:** Readers are never blocked by writers; they simply read a specific snapshot version.
- **Expiration:** `expire_snapshots` is critical for production to delete physically removed data files and manifest files.

DO.jpg

Delta Lake vs Apache Iceberg: Technical Summary

Feature	Delta Lake 3.x	Apache Iceberg 1.5
Metadata Storage	JSON Transaction Log	Avro Manifests + Snapshots
Schema Evolution	Add/Overwrite only	Rename, Reorder, Drop
Partition Evolution	Full Table Rewrite	Yes, Zero-Rewrite
Column Identity	By Name (String)	By Integer ID (Stable)
Row-level Deletes	Rewrite Partition File	Delete Files (v2 Spec)
Engine Support	Spark / Databricks first	Trino, Flink, Athena, Spark
UPSERT / MERGE	Excellent / Mature	Good (Improving)
Streaming (Sink/Source)	Excellent (Spark-native)	Good
Catalog Ecosystem	Unity Catalog, HMS	REST, Nessie, Glue, Hadoop
Core Strength	Spark-Centric Speed	Multi-Engine Flexibility

Choose Delta Lake: If your stack is Spark/Databricks heavy and you need the most mature MERGE/Streaming performance.

Choose Iceberg: For cloud-native architectures where Trino, Flink, and Spark share the same data with zero lock-in.

DO.jpg

Task Overview (100 Marks Total):

1. **Custom Domain Table (25m):** Create an Iceberg table (5k+ rows, non-sensor data) on MinIO. Use a timestamp partition transform and run 3 SQL analytics queries.
2. **Evolution Practice (25m):** Perform ADD column, RENAME column, and a PARTITION transform change. Capture the MinIO folder structure showing old/new partitions coexisting.
3. **Snapshot Rollback (20m):** Generate 3 snapshots. Compare row counts between Snapshot 0 and Current. Execute a successful rollback to Snapshot 0.
4. **Delta vs. Iceberg Benchmarking (20m):** Write the same data to both formats. Compare GROUP BY execution times and physical plans (explain()). Include a 250-word report.
5. **Technical Reflection (10m):** Discuss a scenario where partition evolution saves compute and your observations on the Iceberg metadata JSON structure.

UPCOMING: Project Proposal (Due Monday 30 March)

Submit a 2-page PDF on Classroom: Team names, Problem statement, **Architecture Diagram**, Data source, and Tech stack.

Expand Your Knowledge: Apache Iceberg Resources

Official Academic:

- **Official Documentation:** iceberg.apache.org
The primary source for spec v2/v3 and configuration.
- **Iceberg Spec v3:** [github/apache/iceberg](https://github.com/apache/iceberg) Deep dive into Puffin files and Deletion Vectors.
- **The Definitive Guide:** dremio.com/resources
The most detailed free architectural ebook available.

Hands-on Practical:

- **Iceberg 101 (Dremio):**
dremio.com/sub-surface Excellent technical "how-to" articles that aren't behind a Databricks redirect.
- **Nessie Documentation:** projectnessie.org
Essential for understanding the Catalog and WAP workflows.

Video Community:

- **Iceberg Summit YouTube:** @ApacheIceberg
Highly recommended: Search for "Snapshot Management" and "Manifest rewrite" sessions.
- **Iceberg Python (Pylceberg):**
py.iceberg.apache.org For those wanting to access Iceberg without a heavy Spark JVM.

Follow the **Apache Polaris (Incubating)** project for the latest in interoperable REST catalogs across Snowflake and Spark.

"Metadata is the key that unlocks the scalability of the Data Lakehouse."

DO.jpg